

第 6 课 · 从 Demo 到企业级服务

让调查、资源与责任都具有可核查的边界

从概念、项目代码到运行证据的连续阅读材料。

目录

第 6 课 · 从 Demo 到企业级服务 3

 本章解决的问题 3

 贯穿案例：一次调查进入服务之后 3

 服务架构：把判断放在受约束的边界内 4

 API 合同：请求、快照与错误 5

 UI 是调查的读者，也是动作的发起者 6

 租户隔离：归属检查与可信身份是两层 7

 并发闸门：保护的是每次模型步骤 7

 限流与配额：四个不同的问题 8

 超时与重试：在剩余预算里安排尝试 9

 熔断、缓存与降级：当前 V2 真正做了什么 10

 四组运行证据：收束成可复核的行为 11

 Trace、隐私和审计责任 14

 部署责任：还需要哪一层工程 14

 常见反例：从观察反推错误假设 16

 本章总结：让成功和失败都可解释 16

 课后复现与阅读路线 16

 思考题 17

第 6 课 · 从 Demo 到企业级服务

让调查、资源与责任都具有可核查的边界

本章解决的问题

我们已经能在终端完成一次 GPU assert 调查。Agent 读取事实、选择工具、检索 Runbook，输出带引用的报告；动作建议经过人工审批，再留下模拟执行记录。把它放到网页上，供许多人同时使用，会出现另一组问题：请求能不能无限排队？一个租户是否会耗尽模型容量？模型返回 503 后应该等待多久？用户点击取消，后台是否还在花钱？审批记录能否在重启后找回来？这些问题不由模型回答得聪明与否决定，却会影响每个用户看到的结果。

企业服务把一次成功的调用变成可持续、可解释的行为。它需要明确接纳什么请求、拒绝什么请求，在资源不足时如何停止，在响应不确定时如何核实，以及谁对外部动作负责。本章从当前 V2 的真实代码出发，讨论已经实现的保护与尚未完成的生产责任。读者应能区分运行边界、测试证据和后续设计要求，而不是从‘有 API、有网页’直接推断系统可以接入真实生产平台。

1. 沿 API、UI、runtime、模型网关、工具和报告追踪一次完整调查。
2. 区分调查并发、模型步骤并发、QPS、token 用量和费用限制。
3. 解释有界队列、等待截止、有限重试、Retry-After 与熔断的关系。
4. 复核完整调查、受控上游故障、人工审批和评测回归四组证据。
5. 理解租户、隐私、缓存、持久化、多实例和真实动作的部署责任。

章节关联：第 2 课 · 总截止时间、步骤预算和停止条件仍由调查状态机维护。

章节关联：第 4 课 · 审批和模拟执行沿用独立动作工作流，网页不能越过服务端策略。

章节关联：第 5 课 · 服务是否可靠和调查是否正确应分别取证，不能用一类测试替代另一类。

贯穿案例：一次调查进入服务之后

案例仍是 case-gpu-assert：job-v2-101 的日志首先出现 GPU device-side assert，后续分布式通信错误可能是连锁反应。前几章关注模型如何收集和解释证据，这一章关注同一调查外部：租户 tenant-01 的分析者从网页提交请求，服务端创建 session，逐次接纳模型步骤，最后返回报告。复核者查看引用并提交 annotate_incident 提案，操作人员在审批通过后确认模拟执行。

同一个案例可以观察四种不同结果。健康路径返回 completed 和报告；模型从第一步失败时返回 stopped，报告为空；未经批准的执行返回冲突；改变代码后运行评测，检查引用和根因是否退化。completed 描述调查阶段，executed 描述审批记录，HTTP 状态描述接口结果。它们不是同一个变量，也不应在界面上被压缩成笼统的‘成功’。

观测基线：2954b1d，2026-10-08。健康路径是人工编写回放（authored replay）；固定 503 验证真实网关保护，不连接供应商，不代表在线能力或生产吞吐量。

源码定位：tests/test_course_content.py · test_chapter_six_complete_api_and_approval_observation_is_repeatable

服务架构：把判断放在受约束的边界内



图 6-1 当前 V2 职责路径；回环表示快照更新，不是自动恢复 API。

build_runtime 构造 InvestigationRuntime，后者拥有模型网关、闸门、限流器、熔断器和进程内 records。每条记录包含自己的 orchestrator、trace recorder 和租户归属。API 调用这些对象，CLI 也复用同一 runtime。因此网页和命令行共享调查语义，而不是各写一套判断。工具 registry 由当前策划案例建立；在线模式让真实模型做判断，工具仍未连接实际训练平台。

源码定位：src/ailab_ops/runtime.py · build_runtime

源码定位：src/ailab_ops/v2_runtime.py · InvestigationRuntime

阅读时要辨认默认路径和历史资产。serving/service.py 是 V1 服务，legacy 命名空间用于显式历史实验。V2 复用了 gate.py、limits.py 和 cache.py 中的熔断器，但没有复用 V1 的答案缓存与确定性证据摘要降级。一个模块存在不等于全部功能已进入默认调用链；是否生效要从 runtime 的构造和调用位置追踪，不能只读模块注释。

源码定位：src/ailab_ops/serving/service.py

源码定位：src/ailab_ops/legacy/runtime.py · build_legacy_runtime

API 合同：请求、快照与错误

POST /v2/investigations 接收 case_id、可选 question、tenant_id、user_id 和预算。当前 case_id 对应三份策划案例，question 可在在线模式补充目标；回放只接受已录制输入。服务校验输入后等待调查到终态或 awaiting_approval，再返回快照。它不是先返回后台任务 ID 的 202 接口，也没有向浏览器逐 token 推送结果。一次长调查会占用一个等待中的 HTTP 请求。

复现命令：在明确配置为 replay 的本地服务运行。

```
curl --no-proxy "*" -sS http://127.0.0.1:8080/v2/investigations \
-H "Content-Type: application/json" \
-d '{"case_id":"case-gpu-assert","tenant_id":"tenant-01"}'
```

源码定位：src/ailab_ops/serving/v2.py · InvestigationBody

接口	可观察的合同
POST /v2/investigations	等待调查结果，返回 session_id、phase、报告、证据、审批和错误
GET /v2/investigations/{id}	读取当前租户的完整调查快照，不继续模型调查
GET .../{id}/evidence	当前会话证据及稳定引用标识
GET .../{id}/timeline?limit=N	最近 N 条事件，默认 100、最大 500，标明截断
GET .../{id}/approvals	读取审批记录，查询时检查过期
GET /v2/health	模式、闸门、熔断和存储边界，不证明真实模型调用成功

session_id 定位调查，trace_id 关联观察记录，evidence_ids 定位事实引用。它们承担不同角色。把一份有 session_id 的报告交给另一条调查，可能留下无法解析的引用；用 trace_id 替代租户授权，也会把观测标识错误地变成访问凭据。

对已创建的失败调查，V2 保留 phase、stop_reason 和 error。error 使用 kind、retryable 和 retry_after_s，不直接把供应商错误正文或带凭据的异常字符串返回浏览器。排队失败、租户限制、回放未命中和模型故障可被界面区分。未创建调查就拒绝的入口 QPS 超限返回限流错误，不能假定每个错误都附带 session_id。

响应	当前含义	如何阅读
200	正常调查快照	检查 phase/report；证据不足也能完成
404	不存在或非当前租户	不披露另一租户会话存在性
409	审批冲突或提案不合法	核查服务端状态，不自动执行
422	输入验证失败或 replay_miss	修正输入，未录制问题不会伪造答案
429	租户用量或入口限流	区分可等待配额与费用上限
503 / 502	可重试 / 不可重试模型失败	查看 kind；当前队列拒绝也走 503

源码定位：src/ailab_ops/serving/v2.py · create_v2_app

UI 是调查的读者，也是动作的发起者

服务根路径提供中文调查工作台，资源来自同源 /static/v2/，没有额外前端构建服务。工作台依次呈现计划、证据、假设、报告、时间线和审批。看报告时先把判断与证据锚点对应，再看排除项和未知信息；看动作时先看参数、理由、风险和回滚，再看状态。UI 让调查关系可阅读，而不只是把最终 answer 放进聊天气泡。

源码定位：src/ailab_ops/serving/static/v2/index.html

源码定位：src/ailab_ops/serving/static/v2/app.js

模型网关向上游请求 SSE，收集完整响应后才交给 orchestrator。浏览器端当前采用 JSON 快照与轮询，非终态每 2.5 秒读取服务端；创建请求等待期间显示 loading。网页里的‘正在调查’可能是浏览器等待真实 POST，而不是已经持久化的 investigating 事件。将上游 SSE 和前端流式体验混在一起，会让人误以为部分工具参数已在浏览器逐片段可见，甚至可以边收到边执行。

localStorage 只保留租户与当前 Session ID。刷新后重新 GET 服务端，不保存证据全集、批准确认、执行确认或操作者。切换租户会清空旧调查与草稿，迟到请求由 generation 校验丢弃。POST 传输异常时，网页保留最后确认快照并禁止继续写入，直到完整刷新成功。这处理的是‘是否写入不确定’，不把网络异常自动等同于写入失败。

取消按钮中止浏览器等待，不能保证服务端停止。服务端协程实际收到取消时，runtime 才设置协作取消标记，等待当前模型步骤收尾、结算 usage，并阻止后续工具与模型调用。

UI 已做本地 replay 的真实浏览器验证：桌面和窄屏七种状态、Tab/Shift+Tab、独立审批确认、200% 等价重排和长文本换行。在线浏览器端到端验证尚未完成，真实供应商、长时响应和不同浏览器仍需各自取证。旧 Chromium 92 用于本机兼容性 QA，不能当成生产浏览器选择建议。

源码定位：docs/ui/v2-investigation-workspace.md

源码定位：scripts/qa_v2_workspace.cjs

租户隔离：归属检查与可信身份是两层

调查创建时保存 `tenant_id`。读取会话、证据、时间线和审批通过 `record()` 检查归属，直接拿到 `approval_id` 也不能绕过调查所有权。另一租户查同一 ID 返回 404。这避免接口向错误租户返回记录，但前提是服务获得的 `tenant_id` 可信。

源码摘录：会话存在性和归属一起检查。

```
record = self.records.get(session_id)
if record is None or record.tenant_id != tenant_id:
    raise KeyError("Unknown investigation")
return record
```

源码定位：src/ailab_ops/v2_runtime.py · record

当前 `tenant_id`、`user_id`、`actor` 是未经认证的模拟身份字段，调用者可以修改它们。因此归属检查不能阻止知道 Session ID 的人冒称同一租户，费用限制也不能阻止他换 `tenant_id` 重新获得配额。把输入框改成下拉框并不改变这个事实，HTTP 请求仍可以直接构造。

真实身份接入需要可信网关或应用认证层验证用户，从可信声明获得租户与权限，传给会话和审批边界。数据库查询应带租户条件，执行端再次核查授权。当前工具读取共享策划案例；已有会话归属不表示真实训练任务的数据权限已实现。真实数据适配器还要落实作业可见范围、证据访问权和动作权限。

术语 · 租户隔离

隔离描述数据与资源归属；认证描述身份为何可信；授权描述这个身份能做什么。三者互相依赖，不能增加一个 `tenant_id` 参数就一次完成。

并发闸门：保护的是每次模型步骤

模型调用可能长时间占用上游。一条调查有多个步骤，如果直接把所有调查交给线程池，上游和本地等待队列会一起膨胀。V2 每次 `advance` 前先等待 `UpstreamGate`，再提交专用线程池；没有先把整个调查塞进默认 `executor`。专用池线程数与闸门容量一致，防止闸门之前出现隐藏模型队列。

源码摘录：队列等待受队列超时和调查剩余时间共同约束。

```
await self.gate.acquire(timeout_s=min(self.settings.queue_timeout_s, remaining))
acquired = True
```

源码定位：src/ailab_ops/v2_runtime.py · `_advance`

槽位覆盖一个 `orchestrator.advance` 步骤：模型请求及有限重试，以及响应引发的当前只读工具处理和用量结算。下一步重新竞争槽位，不是用户从头到尾独占容量。`finally` 路径确保退出 `release`。释放前结算 `usage`，再允许下一位复核 `token` 与费用，避免刚排到的请求继续使用尚未入账的配额。

容量限制 `in_flight`，队列限制 `queued`。默认有 8 个步骤槽位、64 个等待位置、30 秒排队超时。满队列立即拒绝，等待超时移除等待者。这些是 `config.py` 的教学默认值，不是经真实压测的容量承诺。调查总截止更短时先受总截止限制；到期还未得到槽位，不发起模型请求。

源码定位：`src/ailab_ops/config.py · Settings`

如果每步平均 10 秒，8 个槽位粗略只能支撑 0.8 步骤/秒。估算忽略响应长度、重试、工具处理和长尾，但已经说明‘能排 64 个就能接 64 个/秒’是错的。队列缓冲尖峰，不创造计算能力。大队列可能只是让更多用户很晚才得知失败。真实容量测试要设置实际工作量，观察等待分布、拒绝率和上游调用数；默认近乎瞬时完成的回放不适合直接测线上吞吐量。

受控实验：容量=1、等待位置=1	观察
A 获得槽位，B 等待	<code>in_flight=1</code> 、 <code>queued=1</code>
C 再申请	<code>queue_full</code> 拒绝，未增加执行数
A 释放给 B	直接移交后 <code>in_flight=1</code> 、 <code>queued=0</code>
B 释放	最终 <code>in_flight=0</code> ； <code>admitted=2</code> ， <code>queue_full=1</code>

该实验使用真实 `UpstreamGate` 和 `asyncio` 任务，没有模型耗时。它验证有界准入与槽位移交，不能用来推算线上吞吐量。`QUEUE_OBSERVATION` 保留稳定统计，章节测试重新执行真实队列比对。

源码定位：`tests/test_course_content.py · test_chapter_six_queue_observation_preserves_admission_and_bounded_queue`

快路径只在信号量有空位且 `heap` 没有等待者时进入，新请求不能越过排队者。队列按优先级、入队时间排序，通用闸门支持 `INTERACTIVE`、`NORMAL`、`BATCH`。当前 `V2 acquire()` 使用默认 `NORMAL`，未按租户配置优先级。API 每秒维护任务把等待至少 5 秒的低优先级请求提升到 `INTERACTIVE`；CLI 没有维护任务。该进程内规则不保证租户均分、跨副本公平或严格 SLA。

源码定位：`src/ailab_ops/serving/gate.py · acquire`

源码定位：`src/ailab_ops/serving/gate.py · try_promote_starved`

取消排队任务可以立即移出 `heap`；已经交给线程的步骤不能强行杀线程。`runtime` 设置 `Event`，等工作收尾，期间仍持有槽位，不能先释放再让新调用超卖容量。闸门还处理取消与移交同时发生的竞态：已移交槽位必须归还下一位，不能丢失。这些 `finally` 和取消路径决定容量统计是否可信。

源码定位：`tests/test_v2_runtime.py · test_cancelled_gate_handoff_returns_slot_to_next_waiter`

限流与配额：四个不同的问题

术语 · QPS、令牌桶与 burst

QPS 是每秒请求数，这里限制用户发起调查的速率。令牌桶（token bucket）把可接纳的请求名额想成桶里的令牌：每个请求取走一枚，系统按 QPS 持续补充，桶满后不再累积。burst 是突发容量，即桶最多保存多少枚。这里的令牌是请求名额，与 LLM 文本 token 不是同一单位。

闸门保护全进程模型容量，RateLimiter 则区分用户、租户和费用。入口 check_request 检查已结算当天费用，占用租户调查并发额度，再检查用户 QPS 与已有 token 窗口。完成或失败均 release_request。租户并发覆盖整条调查，包括排队；闸门并发覆盖当前步骤。某租户暂时没有模型调用，也可能因为很多等待调查而达到租户上限。

控制	当前方式	边界
用户 QPS	tenant+user 的 token bucket，允许 burst	不同用户仍可占满同一租户
租户并发	入口计数整条调查，超额拒绝	限制请求数，不是公平调度
token/分钟	60 秒窗口，每步估算上下文与输出预算	不是原子预留账本
USD/天	已结算消耗与配置单价，UTC 日期重置	不是供应商账单或费用预授权

源码定位：src/ailab_ops/serving/limits.py · RateLimiter

默认用户 QPS=2，burst=4，因此两次快速点击未必被拒绝。租户并发默认 16，token/分钟默认200000，费用/天默认 5 美元。回放费用结算为 0；在线费用按配置的输入、输出单价估算。这些数值用于展示机制，生产设置应根据真实供应商、工作负载和合同确定，不能把配置单价当作供应商报价。

排队前检查还不够，等待期间前面的步骤可能结算用量，所以获得槽位后再次检查。费用超限不建议立即重试，token 窗口超限提供等待提示。不过多个并发步骤可能同时通过检查，没有为各自预留 token或美元，结算仍可能越过限额。当前是预算防护，不是严格原子扣款；多实例需要共享账本与预留/结算规则。

源码定位：src/ailab_ops/v2_runtime.py · _advance

源码定位：tests/test_v2_runtime.py · test_cost_is_rechecked_after_queue_wait_before_model_start

术语 · 限流与预算

限流限制到达速率或并发使用，预算限制一次调查或租户的资源消费。同一个 429 可以有不同原因；费用耗尽未必等待几秒就能恢复。

超时与重试：在剩余预算里安排尝试

调查总截止、队列超时和 HTTP 连接/读写超时位于不同层次。总截止覆盖整条调查；排队取 queue_timeout 和剩余时间较小值；网关每次尝试又取请求 timeout 和 remaining 较小值。默认 HTTP timeout=60 秒、连接超时=5 秒，API 默认调查总时间=120 秒。连接成功不表示模型能在总截止前输出完整结果，长 SSE 读取同样要控制。

源码定位：src/ailab_ops/models/openai.py · complete_with_control

源码定位：src/ailab_ops/models/protocol.py · ModelRequestControl

同步 HTTP 与工具处理不能在任意时刻强制中断。控制在尝试前、失败后、退避等待和响应处理边界检查截止/取消，HTTP timeout 限制网络等待。它阻止过期后创建新尝试或执行后续工具，不表示能精确在某毫秒终止已运行线程或上游计算。判断取消是否省钱，要同时看服务端状态、结算 usage 和供应商执行。

OpenAIModelGateway 默认 max_retries=2，即初次请求加两次重试，共最多三次。连接/读取等网络失败、临时 429、408 和部分 5xx 可有限重试；401、403、普通 4xx、配额型 429 和协议错误不盲目重试。429 含 quota 等信息会归类为不可重试配额错误。该分类影响是否继续，不能把所有非 200 都归为模型能力。

上游有效 Retry-After 优先，没有有效提示才使用指数退避与抖动。延迟被 retry_max_delay_s 截住，默认 30 秒，负数或非有限数不会无限传给 sleep。runtime 中退避经 ModelRequestControl.wait 检查取消与剩余时间；等待超出总截止会停止，不能为了尊重上游突破自身预算。

源码摘录：最后一次失败不再等待；受控调查采用可中断退避。

```
if not error.retryable or attempt == self.max_retries:
    raise error from None
if control:
    control.wait(error.retry_after_s)
else:
    time.sleep(error.retry_after_s)
```

源码定位：src/ailab_ops/models/openai.py · complete_with_control

API 给浏览器的 Retry-After 是另一层提示：秒数向上取整，至少 1 秒。本章受控上游提示 0，目的是不真实睡眠，失败 API 仍返回 Retry-After: 1。客户端可据此安排新请求，但当前网页不自动再次 POST。动作请求尤其应先查审批和执行结果，避免把响应丢失误认成动作未发生。

源码定位：src/ailab_ops/serving/v2.py · _retry_headers

如果网关最多三次，服务又重跑三次，浏览器再重复三次，最坏会产生 27 次上游请求。单层看都有限，组合后却放大故障。V2 网关拥有网络重试，runtime 不自动重跑 stopped 调查，UI 不自动重发写入。第 2 章的报告校验修正是另一类有限模型步骤，仍受步骤、token、时间预算约束。设计客户端要写清重试层次，而不是每个 catch 都补循环。

熔断、缓存与降级：当前 V2 真正做了什么

重试处理偶发失败，熔断处理持续失败。GuardedGateway 使用名为 model 的全进程共享 CircuitBreaker，默认连续五次可重试完整网关调用失败后打开，冷却 20 秒，再允许一个半开探测。一次失败是有限重试全部用完的 complete 调用，不是每个 HTTP 503；五次失败因此可产生十五次尝试。不可重试用户错误不会按同样方式累计上游不健康。

源码定位：src/ailab_ops/v2_runtime.py · GuardedGateway

源码定位：src/ailab_ops/serving/cache.py · CircuitBreaker

打开后新模型步骤不发送上游请求，返回 circuit_open，health 显示 degraded。health 不主动探测模型，未打开熔断时即使刚失败几次也可能是 ok。共享熔断减少同时探测坏上游，但一组租户的可重试失败也可能影响其他租户。隔离范围需要结合上游路由与故障归因设计，不能把进程状态当成集群共识。

V1 SemanticCache 有租户命名空间、TTL 和近似问题匹配，作为历史资产留在 cache.py。V2 health 明确写出 disabled: session-scoped evidence，runtime 没有构造答案缓存；历史 semantic_cache_enabled 默认真也不改变 V2。网页刷新读 records 是查快照，不是新调查缓存命中。

缓存引用报告比缓存字符串更困难。新调查证据 ID、租户、数据版本与原调查可能不同。只用相似问题为键，旧日志判断可能复用于已经变化的作业。后续安全缓存应规定租户、数据/工具/模型/策略版本、证据迁移和引用重新校验。这些语义完成之前，禁用答案缓存保留可读、可验证的会话事实。

模型失败时停止调查，保留已有证据、未知项、预算与停止原因。首步失败可能没有证据，后来失败应阅读已有事实。当前没有自动切换 authored replay，没有自动生成确定性根因，没有 V1 的 evidence_only 摘要降级，也没有公开恢复 API。GET 只读旧状态，新 POST 创建新调查。用户可以另启明确配置 replay 的服务看录制案例，它不是失败在线会话的智能续跑。

规格中的‘保存可恢复状态’是目标。当前只保留进程内 stopped 状态，未完成跨进程恢复、后台恢复任务或恢复 API。教材按实现边界说明，不能把目标写成交付行为。

后续若加证据摘要降级，应明确它由确定性整理产生，保留引用与未知项，不趁模型故障重新引入规则裁决根因。若加持久化恢复，还要定义消息、工具幂等键、审批与预算如何恢复，重新验证权限和事实新鲜度。恢复把运行接回合法状态，并不免除证据是否过期的判断。

四组运行证据：收束成可复核的行为

术语 · 证据一：完整调查

用同一 API 核对步骤、报告、证据、租户归属与时间线窗口。

真实 TestClient 调用 API，模式显式 replay、入口 user_qps=0 以隔离测试。返回 200、completed、gpu_device_assert，四份证据为任务、日志、指标和独立 Runbook。steps_used=5、tokens_used=1000，health 中 gate.admitted=5、in_flight=0。一条 HTTP 调查对应五次步骤准入，admitted 不能当用户请求数。token 来自 authored replay 录制 usage，不是供应商账单。

观测投影：稳定字段；ID、时间戳和局部延迟每次变化。

```
{
  "phase": "completed",
  "steps_used": 5,
  "tokens_used": 1000,
  "root_cause": "gpu_device_assert",
  "evidence_count": 4
}
```

随后用 another-tenant 查询同一 session，返回 404。timeline?limit=2 返回两条、truncated=true，验证接口声明窗口截断，没有冒充完整审计。SERVICE_OBSERVATION 保存观测值，章节测试重新运行 API 比对。

源码定位：tests/test_course_content.py · test_chapter_six_complete_api_and_approval_observation_is_repeatable

术语 · 证据二：上游失败

只替换外部 HTTP 传输，观察真实网关重试和 runtime 熔断。

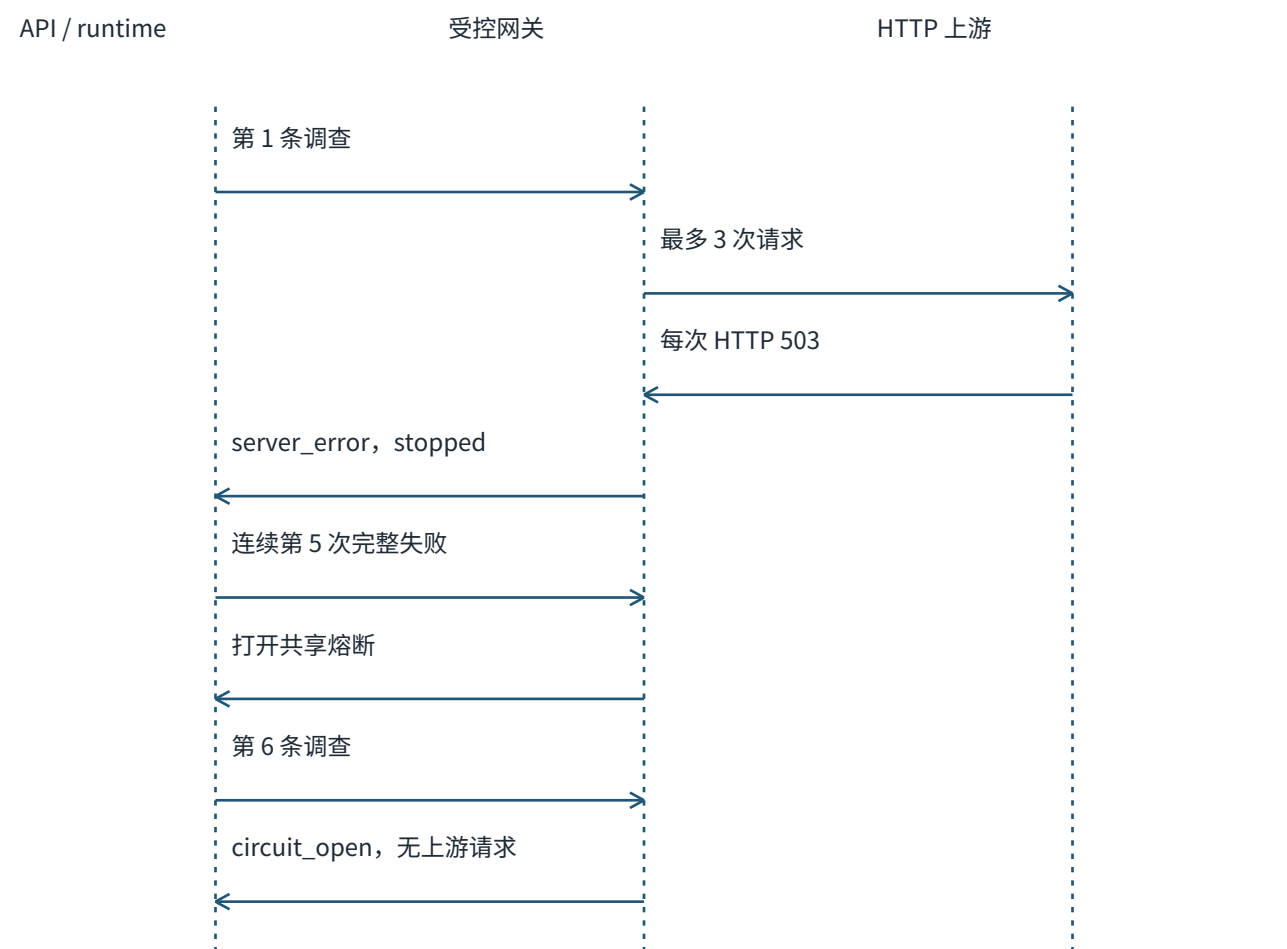


图 6-2 固定 503 故障；‘最多 3 次’ 压缩表示网关有限重试。

传输层固定返回 HTTP 503 和 Retry-After: 0，保留真实解析、分类、重试、GuardedGateway、runtime 和 API，没有 mock 掉 complete 的保护逻辑。六条请求在 20 秒冷却前完成，前五条各尝试三次，第六条不发送上游。每条返回 503、stopped、report=null，首步没有成功所以证据数为 0。

调查序号	error.kind	累计上游尝试	API Retry-After
1	server_error	3	1 秒
2	server_error	6	1 秒
3	server_error	9	1 秒
4	server_error	12	1 秒
5	server_error	15	1 秒
6	circuit_open	15	20 秒

最后 health.status=degraded、breaker.state=open、failures=5, in_flight=0。每个失败 Session ID 都可 GET 到 mode=online 的 stopped 快照，没有伪装成回放成功。停止原因分别为 model_backend:server_error 和 model_backend:circuit_open。这支持重试有界、持续失败不再冲击上游、退出没有漏槽位三项结论，没有测量真实网络长尾、在线模型可用性或吞吐量。

源码定位：tests/test_course_content.py · test_chapter_six_upstream_failure_uses_real_retry_breaker_and_public_error

术语 · 证据三：审批闭环

先批准，再单独模拟执行；核查报告与执行结果是独立对象。

完整调查上提出 annotate_incident，提交当前 evidence_ids、理由、风险和回滚。直接执行 pending 得到 409；批准为 approved 仍无动作结果；单独 execute 才变 executed，simulated=true。原报告不变。‘同意计划’和‘现在执行’分成两个 HTTP 操作，网页也要求两个独立确认。

动作 API 顺序：占位符替换为服务返回的 ID，不能跳过审批。

```
POST /v2/investigations/{session}/approvals
POST /v2/approvals/{request}/approve
POST /v2/approvals/{request}/execute
```

当前没有真实注释写入、作业重启或节点隔离。approved_by 和执行 actor 是模拟字段，未建立实名责任链。第 4 章说明拒绝、过期和重复执行；本章强调发布服务后，可信身份、持久审计与执行器必须共同承担责任。审批卡片不能建立外部权限。

源码定位：src/ailab_ops/approvals/service.py · ApprovalService

术语 · 证据四：评测回归

回放质量、服务保护与在线能力需要不同证据。

分别复核质量回归、服务保护和章节观测。

```
PYTHONPATH=src python3 -m ailab_ops.cli eval --mode replay
PYTHONPATH=src python3 -m pytest tests/test_v2_runtime.py -q
PYTHONPATH=src python3 -m pytest tests/test_course_content.py -k chapter_six -q
```

eval --mode replay 默认三案例各一次，runs=3；root_cause 与 citation_validity 均值为 1.0，tool_choice.mean=null，因为标签未配置要求。provenance 为 authored replay simulation; not model capability。结果验证回放与评分器集成未退化，不证明在线泛化或高负载保护。服务需要故障/队列实验，模型改进需要第 5 章的真实模型重复评测。

源码定位：tests/test_course_content.py · test_chapter_six_regression_summary_matches_current_cli

Trace、隐私和审计责任

trace 将最终答案展开为可追溯事件：模型请求与响应元信息、工具调用、证据新增、状态变化、预算和审批。工具事件可带关联证据 ID，usage/latency 是响应统计，error 说明失败类型。它们让读者区分没查到事实、没解析完整响应、证据校验挡住报告，而不依赖模型私密推理。记录过程不等于记录全部模型消息或 chain of thought。

源码定位：src/ailab_ops/observability/events.py · TraceEvent

TraceRecorder 在保留或导出前复制并递归脱敏：凭据字段、已配置密钥、Bearer/Basic 文本与嵌套 JSON 都处理，tokens_in/out 数值仍保留。API presentation 另加私密推理、原始响应、完整 messages 与评测标签敏感字段。UI 用 textContent 渲染动态内容，证据中的 HTML 不会执行。

源码摘录：列表内的对象也递归脱敏，避免只检查顶层字段。

```
if isinstance(value, (list, tuple)):
    return [self.redact(item) for item in value]
```

源码定位：src/ailab_ops/observability/recorder.py · redact

脱敏按字段、已知值与凭据形状过滤，不理解所有企业敏感内容。日志仍可能含客户名称、工作目录或业务文本，内部原始状态和用户输入也不是全部先清洗再存储。生产还需数据最小化、访问控制、保留期限与脱敏验证。trace 当前在进程内，write_jsonl 是显式导出能力，服务不会自动写可靠审计存储。

源码定位：src/ailab_ops/observability/recorder.py · write_jsonl

时间线默认最近 100 条、最多 500 条，truncated 表示更早事件未返回。这限制响应体，不限制 recorder 累积事件与 records 会话总数；当前没有会话淘汰、总内存上限或全历史分页服务。多实例审计要能跨副本关联、持久保存并按权限查询。把网页截断时间线截图当完整审计，会遗漏背景。

另一个易误读字段是 retry。当前模型事件中的 retry 主要来自报告校验修正次数，网关内部网络尝试未逐次形成独立 trace；usage 也只结算成功获得的响应，不能补记供应商未返回 usage 的失败尝试。要分析在线重试成本，应补请求尝试标识和供应商账单，不从现有字段猜三次 HTTP 的全部 token。

源码定位：src/ailab_ops/investigation/orchestrator.py · advance

部署责任：还需要哪一层工程

当前复现部署是一份本地单进程服务，绑定 127.0.0.1，通过同源页面访问。密钥只进服务配置，不进浏览器；默认 online 要求配置，replay 显式启用。能启动只是第一项事实；扩大访问范围需要回到身份、资源、状态和动作四条边界。

离线服务：打开 `http://127.0.0.1:8080/`，确认回放标记。

```
AILAB_MODEL_MODE=replay PYTHONPATH=src python3 -m ailab_ops.cli \
    serve --host 127.0.0.1 --port 8080
```

在线路径在服务进程配置 `AILAB_MODEL_MODE=online`、`AILAB_LLM_BASE_URL`、`AILAB_LLM_MODEL`与 `AILAB_LLM_API_KEY`，再运行同一 `serve` 命令。密钥从私有配置注入，不写进教材、源码或浏览器。先用一条调查确认协议、工具调用、`usage` 与停止原因，再验证浏览器长时体验和故障。不能用`health.model_mode=online` 代替端到端结果；接口启动和供应商可调用是两个证据。

当前事实	生产前需补足的责任
调查、证据、审批、trace 在进程内	持久化与版本；恢复消息、预算和幂等状态；保留/淘汰策略
单进程闸门、限流、熔断	多实例共享配额和上游容量；全局准入、公平与探测
模拟 tenant/user/actor，无认证	可信身份、会话授权、实名审批和执行权限
策划数据和模拟动作	真实数据适配器、最小权限执行器、外部幂等/补偿和审计
本地 replay 浏览器与集成 QA	在线浏览器验证、长时断连、负载、重启和恢复
脱敏副本和局部用量统计	数据治理、审计存储、尝试级观测与账单对账

直接把进程数 1 改成 4 不是透明扩容。每实例都有 records、审批、熔断与 8 个槽位，总体可能放行 32 步骤；POST 在 A 创建，GET 到 B 会找不到记录。黏性会话能缓解路由，但不能救回 A 重启后的状态，不能将本地额度自动变成全局额度。本书不把增加 worker 当成完成多实例方案。

真实动作还引入分布式问题：外部系统完成后，服务可能在写审计前崩溃。再次执行是否产生第二次副作用？进程内记录只在当前生命周期记住结果，不能跨崩溃承诺 `exactly-once`。需要外部幂等键、持久决策与结果核验，必要时补偿部分成功。真实执行责任属于明确授权的执行器和操作流程，不属于模型生成的一句‘已完成’。

源码定位：`src/ailab_ops/approvals/service.py` • `execute`

部署还需规定 `graceful shutdown` 期限、网络出口、请求体上限和日志保留。当前 `close()` 等线程池收尾，没有跨服务排空与恢复协议。容量要来自目标模型受控测试，时间预算包含网络等待与退避，身份权限来自可信边界。这些是接入企业时需负责的工程选择，并非本书已替实际组织完成的配置。

常见反例：从观察反推错误假设

说法	反例与要核查的证据
async API 有无限吞吐量	HTTP 可异步等待，上游容量仍有限；看 in_flight/queued 和调用数
QPS=2 就最多两条调查并行	QPS 管到达速率；慢调查跨多秒，需要并发限制
Retry-After 之后一定成功	只是等待提示；探测仍可失败，费用额度未必恢复
缓存开关真，V2 就有缓存	默认链没有 SemanticCache，health 明确关闭
取消浏览器等待就没有费用	后端未必收到取消，已运行请求可产生 usage
404 证明身份安全完成	归属依赖自报 tenant，未经认证不能建立可信身份
批准就已执行	approved 后单独 execute；教学结果仅 simulated=true
回放满分就能上线	确定性集成之外，能力、负载与恢复需独立证据

反例都把局部机制扩大成另一层保证。纠正时应找出保证在哪个对象、调用链、测试条件下成立。字段叫 tenant_id 不表示可信身份，方法叫 cache 不表示被调用，一次成功不表示失败会释放资源。阅读服务持续追问‘在什么条件下观察到什么行为’，再检查越界条件。

本章总结：让成功和失败都可解释

API 与 CLI 进入共享 runtime，每次模型步骤经过准入、预算复核和受控网关；证据、报告由代码校验，动作由独立审批管理。UI 组织服务端事实，在不确定写入后要求核实。大模型负责判断的地方没有被工程控制替代，工程边界也没有交给模型语言承诺。

1. 闸门限制模型步骤，租户并发覆盖整条调查；QPS、token、费用是不同维度。
2. 有界队列缓冲尖峰，总截止/HTTP timeout 限制等待；有限重试和熔断减少故障放大。
3. 答案缓存关闭，失败保留 stopped 快照；无自动回放降级或公开恢复 API。
4. 归属检查贯穿查询与审批，但身份仍模拟，状态/配额/审计仍进程内。
5. 分别取证调查、上游失败、审批和评测；回放质量不等于在线能力或生产容量。

全书走到这里，Agent 包含数据、判断、控制、证据、安全、评测与服务责任。课后模仿可以先复用职责分解，再选择自己的只读业务场景。真正可复用的是结论可追溯、副作用有授权、失败可解释；项目规模可以不同，这些关系应清楚。

课后复现与阅读路线

路线以已有代码为材料，课后按兴趣逐步阅读与运行。课堂带读不要求现场写代码，不设置必须完成的学生实验。每一步先观察，再读解释结果的最短路径，最后说明仍未验证的边界。

1. 运行 `investigate --case case-gpu-assert --mode replay`，核对 completed、四份证据、五步和 1000 token，回读第 1、3 章。

2. 启动本地 replay 服务，读证据/报告/未知项；刷新会话，核查 localStorage 只存 ID，回读 app.js 恢复和错误处理。
3. 提出 annotate_incident，批准与单独模拟执行，核对报告未改、executed 审计与 simulated=true，回读第 4 章。
4. 运行 chapter_six 测试，阅读固定 503 场景；解释十五次尝试、五次失败、共享熔断和第六次无上游调用。
5. 读 test_v2_runtime.py 的排队截止、取消移交和费用复核，区分入口拒绝、闸门拒绝、预算停止。
6. 运行 eval --mode replay，核对三条运行、1.0 和 null；按第 5 章说明还需要哪些在线重复评测。

课后复现：不需要 API Key，结果始终标记 authored replay。

```
PYTHONPATH=src python3 -m ailab_ops.cli investigate \
  --case case-gpu-assert --mode replay
PYTHONPATH=src python3 -m ailab_ops.cli eval --mode replay
```

模仿新场景时，先写不含标签答案的只读案例与知识材料，定义证据，再配置真实模型和诚实标记的回放。不要先接写操作。等引用、未知项、失败和预算都能说明，再设计可信审批与最小权限执行器。这样每次增加功能都有可检查的输入、输出与责任边界。

源码定位：README.md

思考题

1. 两租户的排队量不同。现有闸门提供什么公平，缺少什么租户级保证？
2. Retry-After: 20，但总时间剩 3 秒，网关怎么处理？新 POST 和旧会话 GET 有什么区别？
3. 旧证据 ID 不在新会话。即使缓存报告根因相同，发布前仍应检查什么？
4. 创建在副本 A、批准在 B、真实动作后 A 崩溃。进程内对象在哪些地方无法支撑责任链？
5. 回放 root_cause=1.0，在线模型长时间无响应。应归因到能力还是服务，还需收集什么证据？